



Microservices and databases

Microservices database challenges 101

A monolithic application interacts with a single database. The data is shared between all application's components. By contrast, in a microservices application, data ownership is decentralized. Every service is autonomous and has its own private data store relevant to its functionality. This means that one service can't modify any data stored inside the other service's database. And that's where the problems start.

No matter what kind of application you build, its microservices need to interact and share data. Because if they don't, you risk having consistency issues like duplicating data. The problem is, in microservices, you can't use ACID transactions for transactions outside a single service. So beware: having private databases instead of a single shared one makes it challenging to implement queries and transactions that span several services.

5 common data-related patterns and 1 anti-pattern

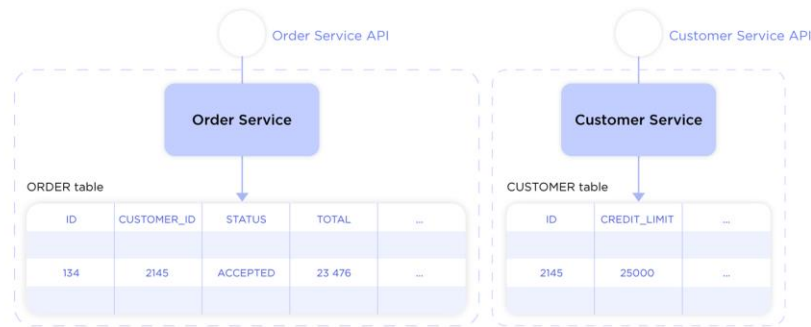
The Database-per-Service pattern

The core characteristic of the microservices architecture is the loose coupling of services. To achieve that, each service must have its own private data store. So, building the database architecture for microservices almost always requires following the database-per-service pattern. At least up till the point where the application hasn't got too complex yet, with numerous services involved.

The service's database can't be accessed directly by other microservices. Each service's persistent data can only be accessed via API.

Let's take a look at an online store application. Order Service and Customer Service each store data in their own databases. Changes to one database don't impact other microservices.

DATABASE PER SERVICE PATTERN



5 common data-related patterns and 1 anti-pattern

The Saga pattern

Using the Database-per-Service pattern creates the need for adopting the Saga pattern as well because sagas are a way of implementing transactions that span services and maintaining data consistency. In microservices, instead of using traditional distributed transactions (XA/2PC-based), you have to use the sequence of local transactions (aka Saga). Here's how it works: one local transaction updates the database and then triggers the next transaction through messaging.

You can choose between two different approaches to coordinating sagas:

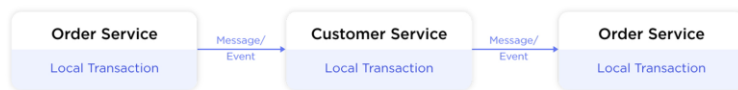
- **choreography** – when exchanging events happens without points of control
- **orchestration** – when you have centralized controllers.

THE SAGA PATTERN

Distributed Transaction (2PC)



Saga

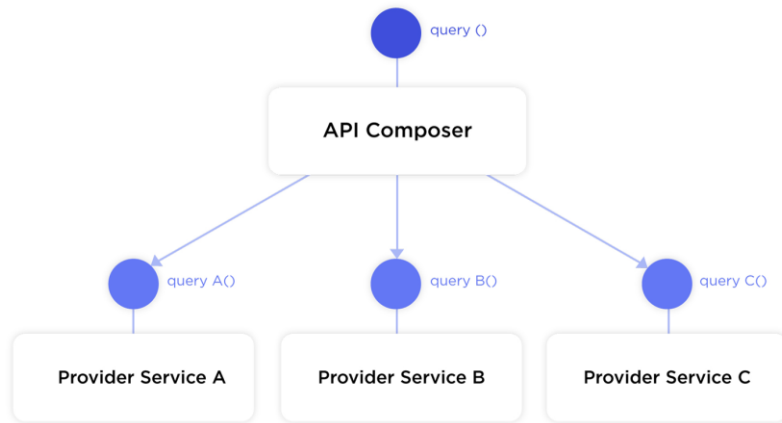


5 common data-related patterns and 1 anti-pattern

The API Composition pattern

Now, let's dive deep into how we can retrieve data that's scattered across multiple services. To do that, you can't use the traditional distributed query mechanism, obviously. But you can use the API Composition pattern instead. It implements a query by invoking certain services that own the data and combining the results. Below, you can see how the query operation is implemented. An API Composer retrieves data from three provider services.

API COMPOSITION PATTERN



5 common data-related patterns and 1 anti-pattern

The CQRS pattern

The command query responsibility segregation (CQRS) is another querying pattern you can use. The thing is, API Composition has some limitations: you can't use it for complex queries because you'll get inefficient in-memory joins. Also, you need to separate concerns to avoid overloading services, which sometimes results in a service that owns data without being the one that should implement the query operation.

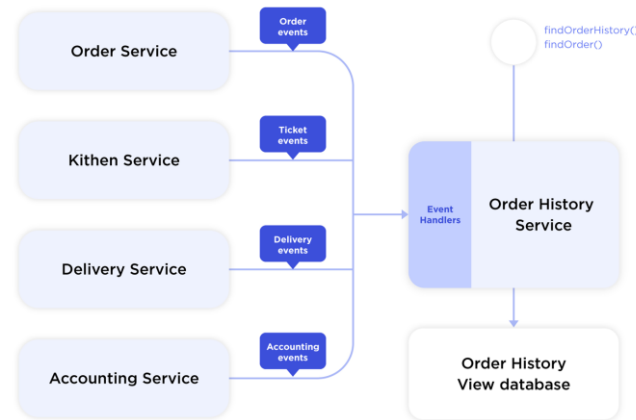
That's when you need CQRS. It implements queries using view databases. CQRS allows separation of concerns. It separates commands from queries. The query-side modules implement queries and keep the data model synchronized with the command-side data model. And as for the command side modules, they manage the operations.

5 common data-related patterns and 1 anti-pattern

The CQRS pattern

You can apply CQRS within a service and use it to define query services. A query service's API consists of query operations only. It queries a database and keeps it up to date by subscribing to events published by the services that own the data.

COMMAND QUERY RESPONSIBILITY SEGREGATION (CQRS) PATTERN



5 common data-related patterns and 1 anti-pattern

The Event Sourcing pattern

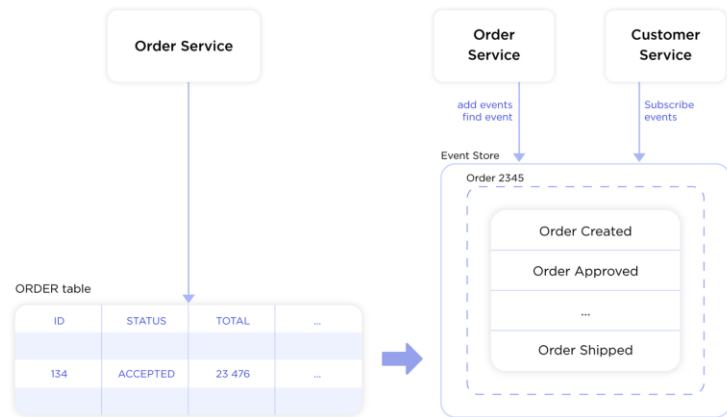
Using Saga creates the need for event sourcing. The Event Sourcing pattern persists aggregates as a sequence of events. When the state of the aggregate changes, a new event is saved to the event store. Then, the application can reconstruct the current state of an entity by replaying the events. An event store uses API to insert and retrieve an aggregate's events. And it also uses API for subscribing to events, similarly to a message broker.

5 common data-related patterns and 1 anti-pattern

The Event Sourcing pattern

Take a look at the application built with event sourcing and the CQRS patterns. Here's how it persists orders. Every order is stored as a sequence of events, and the customer store subscribes to them.

EVENT SOURCING PATTERN



5 common data-related patterns and 1 anti-pattern

The Shared Database anti-pattern

Yes, it's possible to integrate a database for microservices. You can create a single shared database with each service accessing data using local ACID transactions. But if you're seriously considering this, stop right there and think twice.

With a single shared database, you risk losing all the best features of microservices like loose coupling and services independency. With one database, it'll take you more time to coordinate services. Even more, services can have run-time problems and block each other. Also, if a shared database goes down, so will the system. Just be sure to make the database resilient.

How to choose a database for microservices

Today, the database market offers plenty of options. But how do you choose the perfect one for your microservices among them? First, start by picking the approach to creating database models. Let's look at the two – polyglot persistence and multi-model databases.

Polyglot persistence

The microservices architecture enables using different kinds of data storing technologies for different services (aka applying polyglot persistence). Put simply, each development team can choose the persistence technology that suits the needs of their service best.

How to choose a database for microservices

Polyglot persistence

Even more, you can design a service that sits on top of more than one database. The question is, what for? You can have several small services instead of a large and complicated one.

RELEVANT

POLYGLOT PERSISTENCE IN MICROSERVICES



How to choose a database for microservices

Multi-model databases

Alternatively, you can use the multi-model database approach. It allows a database to support more than one abstraction. In other words, you can have one multi-model database but different data models for each service, such as key-value, graph, tabular, etc.

For instance, look at an enterprise application. We have an app with Cassandra's data model (partitioned row store) in the core and DSE Graph on top of it. There's one database but different abstractions on top of the DSE Core for different services.

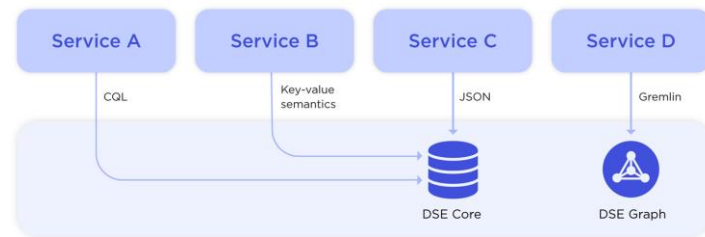
How to choose a database for microservices

Multi-model databases

The multi-model approach offers what polyglot persistence can't – operational simplicity. When you only have one platform, it's easier to manage the system even if each service has its own model of interacting with data. On the other hand, polyglot persistence fits microservices perfectly, and if your application isn't too complex, it's better to stick with it. The good news is you can actually combine both approaches within several services.

Now, you've reached the point where you should choose a database technology for microservices. How? Just ask yourself what kind of data you need to store and how you plan to support microservices data management.

AN ENTERPRISE APPLICATION WITH A MULTI-MODEL DATABASE



How to choose a database for microservices

Non-relational databases

Let's look at some of the non-relational databases(NoSQL) that correspond to different types of stored data and data models. In microservices, you can combine several of those.

Document databases

The databases of this type store and query data as JSON-like documents. Document databases offer an intuitive data model. In other words, documents map to the objects in the application code. This means that you don't have to run JOINS or decompose data across tables. Also, since document databases are distributed systems, they are scalable.

Document databases are perfect for content management and storing catalogs. For instance, you can apply it in a blog application or an e-commerce platform. Popular document databases are MongoDB and Cloudant.

How to choose a database for microservices

Key-value databases

A key-value database stores data using the key-value method. It means that in a key-value store, data is represented as a group of key-value pairs, where the “key” is what identifies them, and the “value” is the data identified. The key-value data store suits session-oriented applications best because it can process large amounts of session-related data fast. The well-known key-value databases are Redis, Amazon DynamoDB, Oracle NoSQL Database.

Graph-based databases

A graph database uses the graph data model in which data entities are connected in nodes and form relationships stored in edges. The main value of a graph is to store and navigate those relationships. We usually apply graph databases in fraud detection, social networks, and recommendation engines. OrientDB, Neo4j, and Amazon Neptune are just a few examples of graph databases you can use.

How to choose a database for microservices

Column-based databases

In a column-based database, data is stored in columns. This means you can access the necessary data more accurately and faster than if it were stored in rows. To do that, you can simply use a column name without scanning the unnecessary information in rows. You can use a column-based database for building a data warehouse. Also, a wide column store is a great solution for Big Data processing. Check out Apache Cassandra or Apache HBase if you're looking for a column-oriented database.

How to choose a database for microservices

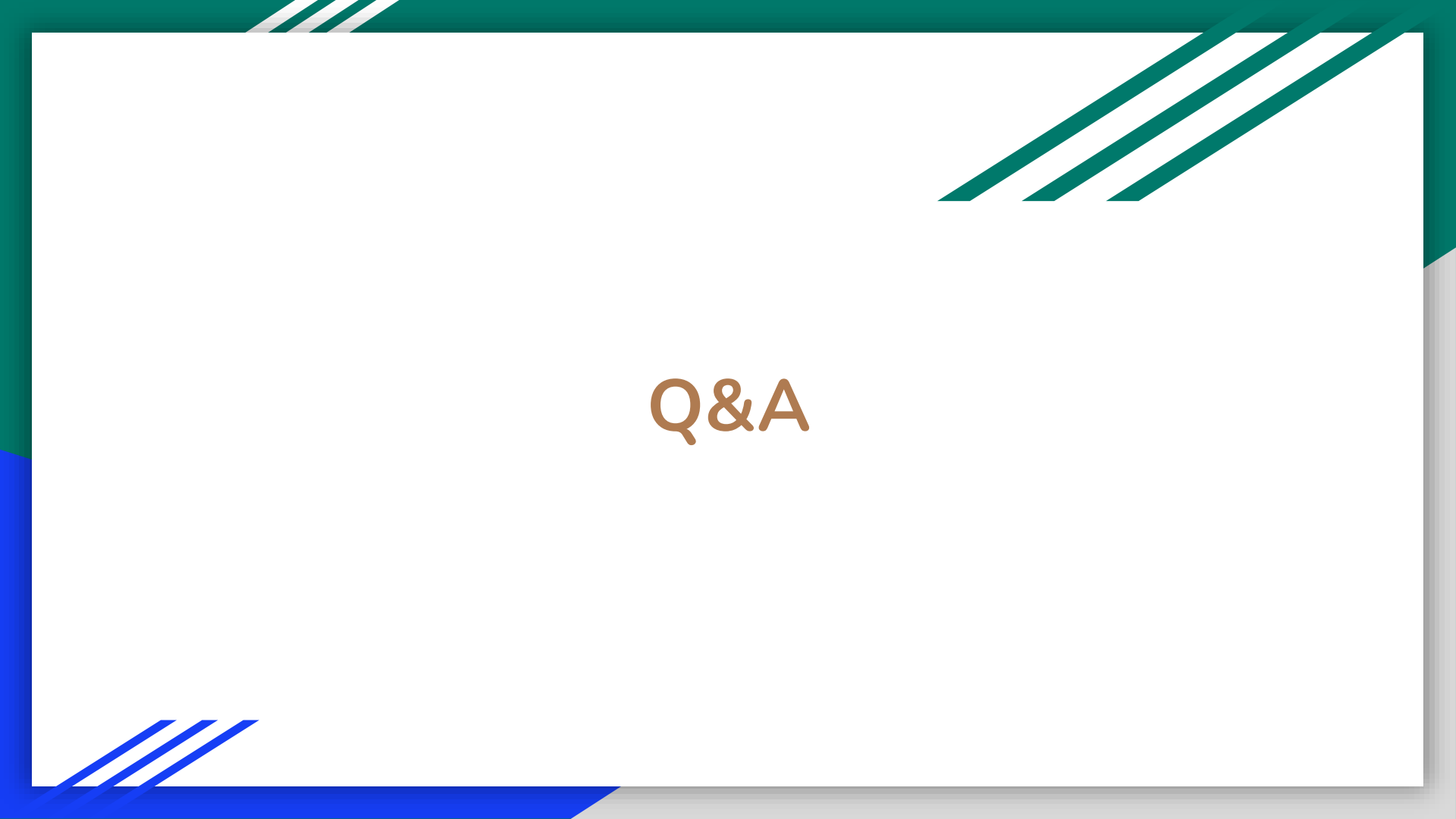
Relational databases

You shouldn't consider using a single relational database if you are building a microservices application from scratch. But you'll have to work with one if your app already exists and it's a monolith. In that case, switching to microservices will mean that you can't abandon the old infrastructure at once. You'll have to start building microservices with what you have – a relational database, such as DB2, MS SQL Server, MySQL, PostgreSQL, and gradually split it into several small services.

On top of that, you can use a relational database in microservices if you apply polyglot persistence. A relational database is an excellent fit for a service that manages low volumes of stable data.

Summary

If you want to have a solid system you need to take care of microservices data management. Start by choosing data-related patterns. Next, you should leverage different approaches to creating data stores and models. Finally, you can pick a microservices database that will match each service's needs best.



Q&A